

```

1. package main
2.
3. import (
4.     "fmt"
5.     "math"
6.     "math/big"
7. )
8.
9. func zsmod(a, b, mod int64) int64 { // 进行模幂运算。
10.     a_ := float64(a)
11.     b_ := float64(b)
12.     mod_ := float64(mod)
13.     var sum float64 = 1
14.     a_ = math.Mod(a_, mod_)
15.     for b_ > 0 {
16.         if math.Mod(b_, 2) == 1 {
17.             sum = math.Mod(sum*a_, mod_)
18.         }
19.         b_ /= 2
20.         a_ = math.Mod(a_*a_, mod_)
21.     }
22.     return int64(sum)
23. }
24.
25. func niyuan(e, fn int64) int64 { // 求逆元。
26.     var s = [3]int64{0, 1, 0}
27.     var t = [3]int64{0, 0, 1}
28.     var o int64
29.     p := fn
30.     for fn%e != 0 {
31.         o = e
32.         s[0] = s[2]
33.         s[2] = s[2]*(fn/e)*(-1) + s[1]
34.         s[1] = s[0]
35.         t[0] = t[2]
36.         t[2] = t[2]*(fn/e)*(-1) + t[1]
37.         t[1] = t[0]
38.         e = fn % e
39.         fn = o
40.     }
41.     for t[2] < 0 {
42.         t[2] += p
43.     }
44.     return t[2]
45. }
46.
47. func zdss(b int64) (int64, int64) { // 求小于 b 的两个最大素数。
48.     var num []int64
49.     for i := b; i >= 2; i-- { // 从 b 开始遍历。

```

```

50.         m := true // m 作为是否为素数的判断依据。
51.         for j := 2; j <= int(math.Sqrt(float64(i))); j++ { // 判断是否为素数。
52.             if i%int64(j) == 0 { // 如果 i 被 j 整除则不是素数, m 置为 false, 并退出循环。
53.                 m = false
54.                 break
55.             }
56.         }
57.         if m { // 若为素数, 则存入切片中。
58.             num = append(num, i)
59.         }
60.         if len(num) == 2 { // 如果已经得到两个素数, 则退出循环。
61.             break
62.         }
63.     }
64.     fmt.Println("不大于 20532175 的两个最大素数为: ", num[0], num[1]) // 输出。
65.     return num[0], num[1]
66. }
67.
68. func lg(a, b int64) { // 求最大公因数和最小公倍数。
69.     var n int64
70.     if a > b { // 选出两个数中最大的一个。
71.         n = b
72.     } else {
73.         n = a
74.     }
75.     var p int64 = 1
76.     for i := n; i >= 2; i-- { // 从 n 遍历到 2, 得到最大公因数即退出循环。
77.         if a%i == 0 && b%i == 0 {
78.             p = i
79.             break
80.         }
81.     }
82.     q := a * b / p // 最小公倍数为两数乘积
                        除以最大公因数。
83.     fmt.Println("\n57123502 与 20532175\n最大公因数为: ", p, "\n最小公倍数为: ", q) // 输出。
84. }
85.
86. func key(p, q int64) (int64, int64, int64) { // 生成公钥和私钥。
87.     var e int64 = 5
88.     fn := (p - 1) * (q - 1)
89.     var d = niyuan(e, fn)
90.     return e, d, p * q
91. }
92.
93. func sol1(y, prikey1, prikey2 int64) { // 化简前的解法。
94.     fmt.Println("化简前解密结果为", zsmo(y, prikey1, prikey2), ".")
95. }
96.

```

```

97. func sol2(y, prikey1, m1, m2 int64) { // 利用中国剩余定理化简后的算法。
98.     b1 := zsmod(y, prikey1, m1)
99.     b2 := zsmod(y, prikey1, m2)
100.    M1 := m2
101.    M2 := m1
102.    M1_ := big.NewInt(niyuan(M1, m1))
103.    M2_ := big.NewInt(niyuan(M2, m2))
104.    m1_ := big.NewInt(b1 * M1)
105.    m1_ = m1_.Mod(m1_.Mul(m1_.Mod(m1_, big.NewInt(m1*m2)), M1_), big.NewInt(m1*m2))
106.    m2_ := big.NewInt(b2 * M2)
107.    m2_ = m2_.Mod(m2_.Mul(m2_.Mod(m2_, big.NewInt(m1*m2)), M2_), big.NewInt(m1*m2))
108.    x := m1_.Add(m1_, m2_)
109.    fmt.Println("化简后解密结果为", x.Mod(x, big.NewInt(m1*m2)), "。")
110. }
111.
112. func main() {
113.     var a int64 = 57123502 // 学号。
114.     var b int64 = 20532175 // 学号倒序。
115.     lg(a, b)
116.     var p, q int64 = zdss(b)
117.     var x int64
118.     fmt.Println("请输入要加密的数字。")
119.     _, _ = fmt.Scanf("%d", &x)
120.     pubkey1, _, pubkey2 := key(p, q)
121.     _, prikey1, prikey2 := key(p, q)
122.     y := zsmod(x, pubkey1, pubkey2) // 加密。
123.     sol1(y, prikey1, prikey2)
124.     sol2(y, prikey1, p, q)
125. }

```

运行结果：

```

> <4 go 设置调用>

57123502 与 20532175
最大公因数为: 17
最小公倍数为: 68992337628050
不大于 20532175 的两个最大素数为: 20532167 20532163
请输入要加密的数字。
123456789123456
化简前解密结果为 123456789123456 。
化简后解密结果为 123456789123456 。

进程 已完成, 退出代码为 0

```